
A Comparative Study of Model Architectures for sEMG-Based Keystroke Recognition

Pranav Akella

Department of Computer Science
University of California, Los Angeles
Los Angeles, CA 90095
pakella44@ucla.edu

Aditya Rao

Department of Computer Science
University of California, Los Angeles
Los Angeles, CA 90095
adityarao4@ucla.edu

Krish Kumar

Department of Computer Science
University of California, Los Angeles
Los Angeles, CA 90095
krishk@ucla.edu

Abstract

This paper investigates the performance of several neural network architectures for keystroke recovery from surface electromyography (sEMG) signals using the emg2qwerty dataset. We benchmark multiple encoder designs, including Baseline (TDSCnv), GRU, GRU2, LSTM, and a hybrid CNN+BiLSTM, with each intended to model the temporal structure present in sEMG data. We also evaluate the effect of training-time data augmentation (SpecAugment, band rotation, temporal jitter) and key input parameters including spectrogram hop length and number of electrode channels. Our results show that the CNN+BiLSTM architecture achieves the best performance with a test Character Error Rate (CER) of 14.83%, outperforming the baseline (25.18%) by 10.35%. We also find that disabling augmentation improves performance in the single-user setting, and that coarser spectrogram resolution (larger hop length) slightly reduces CER. Finally, increasing the number of electrodes significantly improves decoding accuracy, indicating that spatial signal coverage is an important factor in sEMG-based text entry systems.

1 Introduction

Minimizing CER in a single-subject setting is practically important: wearable sEMG systems are typically used by one user, and calibration data are limited, so identifying which modeling choices improve performance under a fixed, small training budget directly informs deployable systems. Thus, in this project, we study modeling choices that most reliably reduce CER for a fixed pipeline using the emg2qwerty codebase. We treat the CTC-based transcription as given and focus on design choices that can be varied within the framework, such as encoder architecture, data augmentation, spectrogram resolution, and number of electrodes.

We compare four encoder variants (GRU, GRU2, CNN+BiLSTM, LSTM) with shared features and greedy CTC decoding so that differences reflect encoder design. The baseline (TDSCnv) and all four variants are trained for 40 epochs to allow a direct comparison. We also test various training-time augmentations (SpecAugment, band rotation, temporal jitter), spectrogram hop lengths (16, 24, 32), and electrode counts (8, 12, 16) to assess how each affects CER under an otherwise identical setup.

2 Methods

2.1 Architectures Overview

We implemented and evaluated several different architectures to test their efficacy for keystroke recovery from sEMG signals. We evaluated each one based on its test CER. All models follow the same high-level pipeline.

2.2 GRU

We chose the GRU as a lightweight recurrent baseline: it has fewer parameters than an LSTM and is widely used for sequence modeling, so it provides a lower-capacity comparison point. The GRU architecture uses a 2-layer bidirectional GRU with a hidden size of 256 and a dropout of 0.1 applied between recurrent layers. It consumes the shared 768-dimensional per-time-step features and produces 512-dimensional bidirectional outputs, which are linearly mapped to character classes. This encoder performs no temporal downsampling, so the emission sequence length matches the input length for CTC alignment.

2.3 GRU2

We chose GRU2 to test whether scaling up the GRU (deeper, wider, with stronger regularization) would improve performance without changing the architecture family, isolating the effect of capacity. GRU2 keeps the same front-end and CTC setup as GRU but increases recurrent capacity to test the effect of scale. The encoder is a 3-layer bidirectional GRU with a hidden size of 384, with a dropout of 0.2 between GRU layers and an additional input dropout of 0.1 applied to the encoder inputs. With bidirectionality, the GRU produces 768-dimensional outputs per time step prior to the character projection head. Like GRU, the GRU2 encoder preserves the full time resolution, allowing direct comparison of recurrent capacity under otherwise matched conditions.

2.4 LSTM

We chose the LSTM as a stronger recurrent baseline than the GRU, with explicit gating for long-range dependencies. The LSTM keeps a separate cell state and distinct forget, input, and output gates, which can improve gradient flow over long sequences. The LSTM architecture replaces the encoder with a bidirectional LSTM-based block designed to preserve the original time resolution while adding recurrent capacity. It uses a 3-layer bidirectional LSTM with a hidden size of 512 and a dropout of 0.3 between layers, producing 1024-dimensional outputs per time step. These outputs are projected back to 768 dimensions to match the front-end feature size, combined with a residual connection, and normalized with LayerNorm before the final character projection. Because there is no striding, the encoder preserves $T_{\text{out}} = T_{\text{in}}$, which keeps CTC alignment comparable to the GRU-based models while changing only the recurrent unit and capacity.

2.5 CNN+BiLSTM

We chose CNN+BiLSTM because the combination of local convolutional features and a bidirectional LSTM is a standard pattern in speech and sequence recognition; the convolutions can capture short-term structure while the BiLSTM models longer-range context. The CNN+BiLSTM model adds a temporal convolutional front-end before the recurrent encoder. After the shared front-end generates 768-dimensional features per time step, the model applies two Conv1D blocks with 256 channels each, a kernel size of 5, and a stride of 2 per layer, with each block using batch normalization and ReLU. The downsampled sequence is then encoded by a 2-layer bidirectional LSTM with a hidden size of 256 and a dropout of 0.1, producing 512-dimensional outputs that are projected to character logits for CTC. This architecture tests whether local temporal aggregation and downsampling via convolutions improves recognition compared to purely recurrent encoders.

2.6 Data Augmentation

Data augmentation was applied exclusively during the training phase using three distinct techniques to enhance model robustness. First, band rotation was implemented by circularly shifting the 16

electrode channels within each band using offsets of -1 , 0 , and $+1$, with the resulting MLP outputs averaged via mean pooling across the three rotations. Second, temporal alignment jitter was used to reduce sensitivity to minor timing errors by applying random temporal offsets of up to 120 samples. Finally, SpecAugment was employed to mask specific time and frequency regions on the log-spectrogram; in the augmentation-enabled CNN+BiLSTM experiment, this involved the application of 5 time masks and 2 frequency masks.

2.7 Experiments

2.7.1 Setup

All runs described below were on 1 GPU with batch size 32 and 2 dataloader workers. The random seed was fixed to 1501. Optimization used Adam with learning rate 10^{-3} , and a linear-warmup cosine-annealing schedule with 10 warmup epochs, warmup start learning rate 10^{-8} , and $\eta_{\min} = 10^{-6}$ over 20 max epochs. For evaluation, we used a CTC greedy decoder and tracked CER, DER, IER, SER, and loss; numerical outcomes are reported in the Results section. The single-user split used user 89335547 with 16 training sessions, 1 validation session, and 1 test session.

2.7.2 CNN+BiLSTM With And Without Augmentation

We ran CNN+BiLSTM (two Conv1D layers with 256 channels, kernel size 5, stride 2, followed by a 2-layer BiLSTM with hidden size 256 and dropout 0.1) in two conditions: with the full augmentation pipeline (band rotation, temporal jitter, and SpecAugment) enabled, and with augmentation disabled. Train, validation, and test used only tensor conversion and log-spectrogram extraction (no band rotation, temporal jitter, or SpecAugment). Model and optimization settings were otherwise identical.

2.7.3 CNN+BiLSTM With Varying Hop Length

To study the effect of spectrogram temporal resolution, we trained CNN+BiLSTM (no augmentation) for 20 epochs with hop lengths of 16, 24, and 32. All other settings matched the no-augmentation run. Larger hop length reduces the number of time frames per second and thus the sequence length presented to the encoder.

2.7.4 Baseline With Varying Number of Electrodes

To assess how the number of electrode channels affects decoding performance, we trained the baseline TDSCnv model for 40 epochs with 8, 12, and 16 electrodes (the full set). The same architecture and optimization schedule were used in all three runs; only the electrode subset and corresponding input dimensionality were changed. This experiment isolates the effect of channel count under a matched training budget.

3 Results

3.1 Model Performance

We evaluated four encoder variants on the same data split. The baseline (TDSCnv), GRU, GRU2, LSTM, and CNN+BiLSTM were each trained for 40 epochs. Table 1 and Figure 1 summarize validation and test CER, along with SER where available.

The baseline TDSCnv model achieved a test CER of 25.18%, providing a reference for the recurrent and hybrid encoders. GRU reached a test CER of 39.81%, while the larger GRU2 reached a test CER of 25.39%. Thus GRU2 substantially outperformed GRU under the same epoch budget, consistent with the benefit of increased capacity (wider and deeper GRU with input dropout). Both GRU and GRU2 remained worse than the baseline on the test set, with GRU2 closer to the TDSCnv baseline.

The CNN+BiLSTM model achieved a test CER of 14.83%, the best among all configurations and 10.35% below the baseline (41.1% relative reduction). The LSTM model achieved a test CER of 19.84%. So among the configurations we ran, CNN+BiLSTM had the lowest test CER (14.83%), followed by LSTM (19.84%) and the baseline TDSCnv (25.18%).

Table 1: Validation and test metrics by model. Baseline from 40 epochs; CNN+BiLSTM from 40 epochs. CER: character error rate; DER: deletion; IER: insertion; SER: substitution (all in %). Epochs indicates training length.

Model	Epochs	Val CER	Test CER	Val SER	Test SER
Baseline (TDSCConv)	40	22.55	25.18	14.51	15.78
GRU	40	28.47	39.81	17.15	21.94
GRU2	40	19.43	25.39	12.16	14.80
LSTM	40	19.54	19.84	12.61	14.48
CNN+BiLSTM	40	14.33	14.83	9.35	10.44

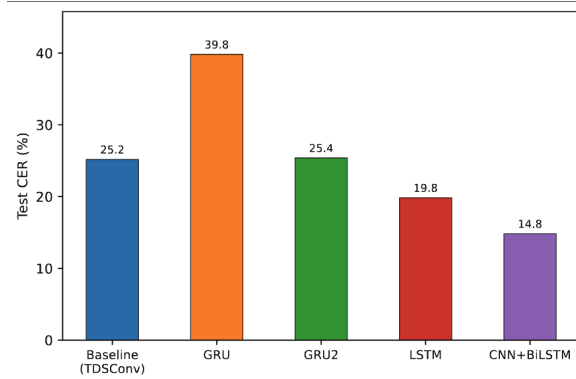


Figure 1: Test CER (%) by model. CNN+BiLSTM achieves the lowest test CER (14.83%), 10.35% below the baseline (25.18%). All models trained for 40 epochs.

3.2 Data Augmentation

We compared two CNN+BiLSTM runs under the same 20-epoch budget: one with the full augmentation pipeline, and one with augmentation explicitly disabled. Both use the same architecture and 20 epochs. Table 2 reports the validation and test metrics for both runs.

With augmentation enabled, the test CER was 21.24%. With augmentation disabled, the test CER was 17.66%. So in this single-user, 20-epoch setting, turning off augmentation improved test CER by roughly 3.5%. The no-augmentation run also had lower validation and test CER, lower validation and test SER, and lower loss.

This result suggests that for this subject and this training length, the added invariance from band rotation, jitter, and SpecAugment did not help and may have hurt by underfitting the data or by regularizing away useful signals before the model could exploit it. Under the conditions we used, the no-augmentation configuration is the better choice for minimizing CER in this setup.

Table 2: CNN+BiLSTM at 20 epochs: with vs. without data augmentation. All metrics in %.

Condition	Val CER	Test CER	Val SER	Test SER	Val loss	Test loss
With augmentation	21.09	21.24	13.36	13.98	0.688	0.68
No augmentation	15.44	17.66	9.88	11.52	0.524	0.573

3.3 Hop length (temporal resolution)

We trained CNN+BiLSTM (no augmentation) for 20 epochs with spectrogram hop length 16, 24, and 32; all other settings were unchanged. Table 3 and Figure 3 report validation and test CER and SER.

Test CER was lowest for hop length 32 (18.18%), followed by hop length 24 (18.98%) and hop length 16 (19.49%). So under this fixed 20-epoch budget, coarser temporal resolution (larger hop

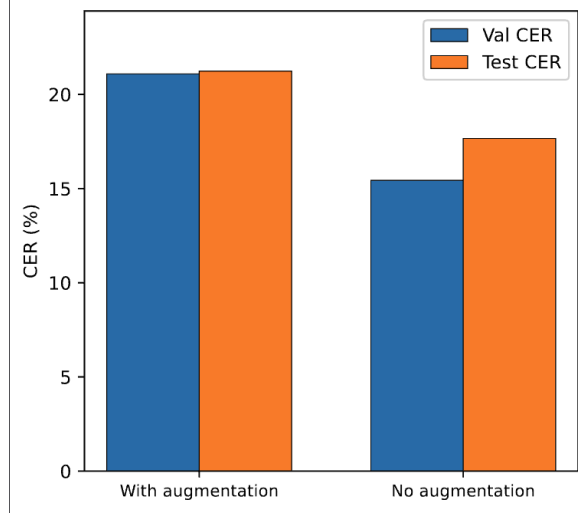


Figure 2: Comparison of validation and test CER for CNN+BiLSTM with vs. without data augmentation (20 epochs. No-augmentation run achieves lower CER on both splits.

length) gave better CER. That may reflect a shorter input sequence for the encoder (fewer time steps), reducing the burden on the recurrent layer, or a regularizing effect of the lower time resolution. We did not sweep other hop lengths or training lengths; the default hop length 16 used elsewhere in the report is finer than the best-performing 32 in this ablation.

Table 3: CNN+BiLSTM (no augmentation) at 20 epochs: effect of spectrogram hop length. All metrics in %.

Hop length	Val CER	Test CER	Val SER	Test SER
16	19.69	19.49	12.43	13.83
24	18.94	18.98	12.18	13.25
32	18.25	18.18	11.92	12.62

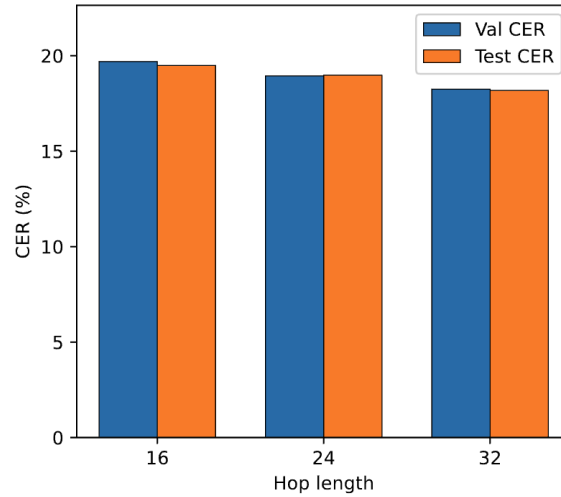


Figure 3: Validation and test CER for CNN+BiLSTM (no augmentation, 20 epochs) vs. spectrogram hop length. Larger hop length (coarser time resolution) yields lower CER in this setup.

3.4 Number of electrodes

We trained the baseline TDSCnv for 40 epochs with 8, 12, and 16 electrodes under otherwise identical settings. Table 4 and Figure 4 report validation and test CER and SER.

Test CER was lowest with 16 electrodes (25.18%), followed by 12 electrodes (27.47%) and 8 electrodes (32.46%). Under this fixed 40-epoch budget, reducing the number of electrode channels consistently worsened decoding: each step from 16 to 12 to 8 added roughly 3–5% to test CER. The same trend held for SER. This indicates that the full 16-electrode configuration provides useful information that is not fully recovered when channels are subsampled, and that electrode count is a meaningful design lever for single-user sEMG-to-text in this pipeline.

Table 4: Baseline TDSCnv: effect of number of electrodes. All metrics in %.

Electrodes	Val CER	Test CER	Val SER	Test SER
8	29.31	32.46	17.21	22.02
12	26.58	27.47	15.31	18.24
16	22.55	25.18	14.51	15.78

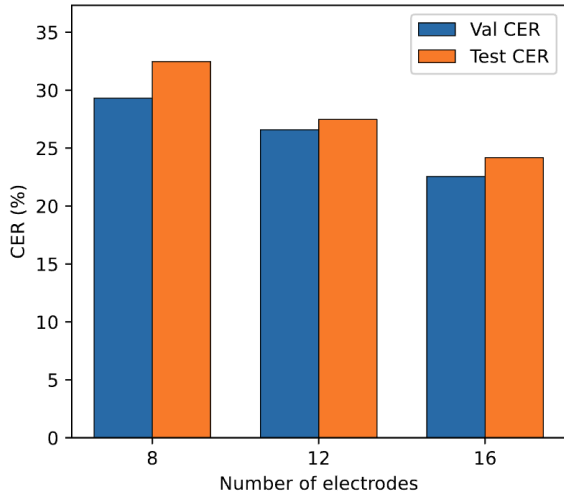


Figure 4: Validation and test CER for baseline TDSCnv vs. number of electrodes. Fewer electrodes yield higher CER.

4 Discussion

4.1 Encoder architecture and capacity

We evaluated five encoder architectures on the same data split and training budget of 40 epochs. Table 1 and Figure 1 summarize the validation and test CER across all models.

The CNN+BiLSTM architecture achieved the best performance with a test CER of 14.83%, substantially outperforming all other models and improving upon the baseline TDSCnv (25.18%). This result suggests that combining convolutional layers with recurrent modeling is particularly effective for sEMG decoding. The convolutional layers aggregate short-term temporal patterns and reduce sequence length through downsampling, which likely produces a more structured representation for the BiLSTM to model longer-range dependencies.

The LSTM encoder also performed well, reaching a test CER of 19.84%. Compared to GRU-based models, the stronger performance of the LSTM may reflect its greater representational capacity and its gating mechanism, which is designed to preserve information over longer time spans. This can be beneficial for sEMG signals, where keystrokes unfold over extended temporal windows.

In contrast, the recurrent-only GRU models performed worse. The smaller GRU reached a test CER of 39.81%, while the larger GRU2 improved to 25.39% but still did not surpass the baseline. Although GRU2 increased capacity, both GRU variants must learn both short-range signal patterns and long-range dependencies through recurrence alone. Without convolutional preprocessing or temporal compression, this makes the learning problem more difficult.

Overall, the results indicate that architectures that either increase recurrent capacity (LSTM) or combine convolutional feature extraction with recurrent modeling (CNN+BiLSTM) are better suited to this task. Among the evaluated models, CNN+BiLSTM provides the strongest performance, suggesting that capturing local temporal structure before sequence modeling is particularly beneficial for sEMG-based keystroke recognition.

4.2 Data augmentation

In our single-subject, 20-epoch setting, training without augmentation (band rotation, temporal jitter, SpecAugment) yielded better validation and test CER than with the full pipeline. Augmentation benefit thus depends on training length and data regime: with short training or single-subject data, regularization can dominate before the model exploits the unaugmented signal.

4.3 Number of electrodes

The electrode-count ablation shows a clear effect, as test CER increased from 25.18% with 16 electrodes to 27.47% with 12 electrodes to 32.46% with 8 electrodes. Thus, reducing the number of channels consistently hurt performance, with each halving of the electrode set adding roughly 3 to 5% to test CER. That suggests the full 16-electrode configuration captures spatial information across the forearm that aids discrimination; subsampling to fewer channels removes information the model cannot recover.

4.4 Temporal resolution (hop length)

Varying the spectrogram hop length showed that coarser temporal resolution (a larger hop) improved test CER: hop 32 gave the lowest CER (18.18%), followed by 24 (18.98%) and the default 16 (19.49%). One interpretation is that a shorter sequence (fewer frames per second) eases optimization or acts as a form of temporal regularization; another is that the default hop 16 was finer than necessary for this task and introduced redundant or noisier structure. In either case, the implication is that temporal resolution is worth tuning rather than leaving at a fixed default.

5 Acknowledgements

We would like to acknowledge Prof. Kao for guidance throughout the quarter through lectures and for providing the project specification. We also thank the course TAs for their support during discussion sections and for answering questions on Piazza.

References

- [1] Awni Hannun, Ann Lee, Qiantong Xu, and Ronan Collobert. Sequence-to-Sequence Speech Recognition with Time-Depth Separable Convolutions. *arXiv preprint arXiv:1904.02619*, 2019. Available at: <https://arxiv.org/abs/1904.02619>
- [2] Alex Graves, Santiago Fernández, Faustino Gomez, and Jürgen Schmidhuber. Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks. *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, volume 148, pages 369–376, 2006. Available at: https://www.cs.toronto.edu/~graves/icml_2006.pdf
- [3] Daniel S. Park, William Chan, Yu Zhang, Chung-Cheng Chiu, Barret Zoph, Ekin D. Cubuk, and Quoc V. Le. SpecAugment: A Simple Data Augmentation Method for Automatic Speech Recognition. *Proceedings of INTERSPEECH 2019*, pages 2613–2617, 2019. Available at: <https://arxiv.org/abs/1904.08779>

- [4] Haohe Liu, Xubo Liu, Qiuqiang Kong, Wenwu Wang, and Mark D. Plumbley. Learning Temporal Resolution in Spectrogram for Audio Classification. *arXiv preprint arXiv:2210.01719*, 2022. Available at: <https://arxiv.org/abs/2210.01719>
- [5] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *Journal of Machine Learning Research*, 15:1929–1958, 2014. Available at: <https://www.jmlr.org/papers/volume15/srivastava14a/srivastava14a.pdf>
- [6] Kyunghyun Cho, Bart van Merriënboer, Çağlar Gülçehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation. *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734, 2014. Available at: <https://arxiv.org/abs/1406.1078>
- [7] Sepp Hochreiter and Jürgen Schmidhuber. Long Short-Term Memory. *Neural Computation*, 9(8):1735–1780, 1997. Available at: <https://doi.org/10.1162/neco.1997.9.8.1735>
- [8] Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling. *arXiv preprint arXiv:1803.01271*, 2018. Available at: <https://arxiv.org/abs/1803.01271>
- [9] Viswanath Sivakumar, Jeffrey Seely, Alan Du, Sean R. Bittner, Adam Berenzweig, Anuoluwapo Bolarinwa, Alexandre Gramfort, and Michael I. Mandel. emg2qwerty: A Large Dataset with Baselines for Touch Typing using Surface Electromyography. *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2024. Available at: <https://arxiv.org/abs/2410.20081>